Università degli Studi di
Milano - Bicocca

Numerical Simulations of the Ising model through Monte Carlo Markov Chains



Maurizio Errico

Supervisor: **Prof. Mattia Bruno**
Course: Fisica (Physics)
Academic Year: 2025/2026
Department: Dipartimento di Fisica "Giuseppe Occhialini"

Abstract

This work presents the numerical study of the ferromagnetic Ising model in one, two, and three dimensions to observe phase transitions as a function of the lattice size. The system configurations were obtained by sampling a Markov Chain at fixed temperature generated with the Metropolis-Hastings algorithm following an analysis of its thermalisation and autocorrelation time. The critical temperature was estimated in the two and three dimensional cases by analyzing the finite size scaling of the magnetization. Analogously, the absence of a finite temperature phase transition was found in the one dimensional case, in accordance with analytical results.

Contents

A brief history of the Ising model **2**

1 Theoretical background **3**
1.1 General overview . . . 3
1.2 Theory of phase transitions . . . 4
1.3 Known solutions . . . 6
1.3.1 One dimensional case . . . 6
1.3.2 Two dimensional case . . . 7
1.3.3 Three dimensional case . . . 8

2 Methods **9**
2.1 Data representation . . . 9
2.2 Monte Carlo Markov Chains . . . 10
2.2.1 Generation . . . 10
2.2.2 Analysis . . . 15
Sokal's method . . . 17
Data blocking method . . . 19
2.2.3 Comparing the two methods . . . 24
2.2.4 Improvement to the MCMC algorithm: sweeps . . . 26

3 Results **29**
3.1 Introduction . . . 29
3.2 1D Ising model . . . 29
3.3 2D Ising model . . . 30
3.4 3D Ising model . . . 32
3.5 General comments . . . 34

A Appendix **36**

A brief history of the Ising model

The Ising model, named after the physicist Ernst Ising, is a mathematical model of ferromagnetism in statistical mechanics. The model was first proposed by Wilhelm Lenz in 1920 and was later solved by Ising (a student of Lenz) in 1925 for the one-dimensional case. The two-dimensional square-lattice Ising model is one of the simplest statistical models to show a phase transition. Though it is a highly simplified model of a magnetic material, the Ising model can still provide qualitative and sometimes quantitative results applicable to real physical systems. The two dimensional square-lattice Ising model is much harder to solve than the one-dimensional case, and was only given an analytic description much later, by Lars Onsager (1944). No complete analytic solution is known for the three-dimensional case, and only numerical methods are available.

The Ising model has been widely studied in physics, mathematics, and computer science, and has applications in various fields.

Chapter 1

Theoretical background

1.1 General overview

The fundamental assumption of the model is that a solid ferromagnet can be represented as an array of fixed ions that only interact with their nearest neighbours (and, optionally, an external magnetic field). Such system is described by the classical Hamiltonian

$$H = -\sum_{\langle i,j \rangle} J_{ij}\sigma_i\sigma_j - \mu\sum_i h_i\sigma_i \tag{1.1}$$

where:

- σ_i is the spin value of the ion of the i-th lattice site.
- J_{ij} is a coupling factor between the i-th and j-th lattice sites.
- The sum over $\langle i, j \rangle$ is meant to be over nearest neighbours.
- h_i is the magnetic field in the i-th site.
- μ is the magnetic moment of the ions.

This is the special case of the more general quantum Hamiltonian

$$\hat{H} = -\sum_{\langle i,j \rangle} J_{ij}\hat{\sigma_i^z}\hat{\sigma_j^z} - \mu\sum_i \left(h_{i,x}\hat{\sigma_i^x} + h_{i,y}\hat{\sigma_i^y} + h_{i,z}\hat{\sigma_i^z}\right) \tag{1.2}$$

where σ_i are replaced by the Pauli matrices of the i-th site.

In this text only $h_{i,z}$ will ever be assumed $\neq 0$. If both $h_{i,x}$ and $h_{i,y}$ are null then every operator commutes with each other in the canonical basis, which means that equations 1.1 and 1.2 are the same.

Given that the quantities N = number of particles, $V = Na^d$ = volume of the lattice (where a is the unimportant lattice constant that can be set to 1) are fixed, if the system is in thermal equilibrium with a reservoir at temperature T then the set of microstates of the system is described by the canonical ensemble. In this case, the probability of the system to be in a microstate σ is given by the Boltzmann distribution

$$P_\beta(\sigma) = \frac{e^{-\beta H(\sigma)}}{Z_\beta} \tag{1.3}$$

where Z_β is the canonical partition function:

$$Z_\beta = \sum_{\{\sigma\}} e^{-\beta H(\sigma)} = \sum_{\{\sigma\}} e^{\beta\left(\sum_{\langle i,j\rangle} J_{ij}\sigma_i\sigma_j + \mu\sum_i h_i\sigma_i\right)} \tag{1.4}$$

In most cases the subscript β will be dropped from the notation.

The minus in front of the first sum in equations 1.1 and 1.2 is conventional. Varying the sign of the coefficients J_{ij} one can describe different types of magnetic interactions:

1. $J_{ij} > 0$: ferromagnetic
2. $J_{ij} < 0$: antiferromagnetic
3. $J_{ij} = 0$: non-interacting

Historically, the Ising model is conventionally intended to be ferromagnetic. For simplicity's sake, J_{ij} along the constants k_B, $\hbar$, h_i, μ will be set $= 1\ \forall i, j$, when not null.

1.2 Theory of phase transitions

It's been stated that the system described by the Ising model is represented by the canonical ensemble, which is governed by the Helmholtz free energy. This, like every other thermodynamic potential, can be expressed as a function of the partition function $Z = Z(T, h)$:

$$F = E - TS = -\frac{1}{\beta}\ln Z \tag{1.5}$$

Formally, a phase transition is defined as a non-analyticity in the free energy in the thermodynamic limit. To observe such a non-analyticity one can study the partial derivative

$$M = -\frac{\partial F}{\partial h} \tag{1.6}$$

which takes the name of magnetization.

This is a handy observable in the numerical approach because its mean per lattice site is easily obtainable by averaging the spin values of the lattice sites. The formula can be derived using equation 1.5 and the definition of the partition function 1.4:

$$\langle M \rangle = \frac{1}{N} \frac{\partial}{\partial h} \left(\frac{1}{\beta} \ln Z \right) = \frac{1}{N} \sum_i \sigma_i \tag{1.7}$$

and studying its behaviour as a function of the temperature T and the external magnetic field h one can identify the phase transitions. Note that the symbol $\langle M \rangle$ in this work refers to the *mean value of the magnetization per lattice site*.

It should be noted that these definitions formally hold even in the case of no external magnetic field. This is true only in the thermodynamic limit, where the system is infinitely large and the magnetization can be non-zero even when h vanishes.

In a finite system the Hamiltonian is symmetric under the transformation $\sigma_i \to -\sigma_i$, and this causes the magnetization to be zero when $h = 0$ because

$$\langle M \rangle = \frac{1}{N} \frac{\partial}{\partial h} \left(\frac{\ln Z}{\beta} \right) \Bigg|_{h=0} = 0 \quad \forall\, T \tag{1.8}$$

The solution to this problem is to consider this partial derivative in the general case, and then take the limit $h \to 0$ after the thermodynamic limit $N \to \infty$. This is equivalent to adding a small external magnetic field to the system, and then taking the limit of this field going to zero after the thermodynamic limit has been taken into account.

Another important partial derivative is the first derivative of the magnetization with respect to the magnetic field, which defines the (isothermal) magnetic susceptibility:

$$\chi = \frac{\partial \langle M \rangle}{\partial h} = -\frac{\partial^2 F}{\partial h^2} \tag{1.9}$$

Similarly to the magnetization, the susceptibility is a fundamental observable in numerical simulations.

Since in the thermodynamic limit phase transitions are characterized by non-analyticities, the behaviour of χ at the critical temperature T_C is a divergence. In a finite system that can be expected to be observed as a peak in the susceptibility at a pseudo-critical temperature $T_C(L)$, which approaches the true critical temperature T_C as the system size L increases, and it's what was used in this work to estimate in practice the critical temperature in the 2D and 3D cases.

1.3 Known solutions

1.3.1 One dimensional case

The one dimensional Ising model is exactly solvable with a method known as the transfer matrix method.

The interaction between adjacent spins can be represented as a symmetric 2×2 matrix, called the transfer matrix, holding the Boltzmann factors of the possible configurations of two adjacent spins:

$$T_{\sigma_i,\sigma_j} = e^{\beta\sigma_i\sigma_j} \tag{1.10}$$

which can be explicitly written by substituting the possible values of σ_i and σ_j:

$$T = \begin{pmatrix} e^{\beta} & e^{-\beta} \\ e^{-\beta} & e^{\beta} \end{pmatrix} \tag{1.11}$$

And if we write the partition function (equation 1.4) as a sum over all the possible configurations of the system, we can see that it can be expressed as a product of transfer matrices (with periodic boundary conditions):

$$Z = \sum_{\{\sigma\}} e^{\beta \sum_{\langle i,j \rangle} \sigma_i\sigma_j} = \sum_{\{\sigma\}} \prod_{\langle i,j \rangle} e^{\beta\sigma_i\sigma_j} = \sum_{\sigma_1,\sigma_2,\ldots,\sigma_N} T_{\sigma_1,\sigma_2} T_{\sigma_2,\sigma_3} \ldots T_{\sigma_N,\sigma_1} \tag{1.12}$$

where $\{\sigma\}$ is the set of all possible configurations of the system.

Using the properties of the trace operator, we can rewrite the partition function as the very elegant:

$$Z = \mathrm{Tr}\left(T^N\right) \tag{1.13}$$

Since the trace of a matrix is equal to the sum of its eigenvalues, one can find Z by diagonalizing the transfer matrix:

$$0 \stackrel{!}{=} \det\left(T - \lambda I\right) = \det \begin{pmatrix} e^{\beta} - \lambda & e^{-\beta} \\ e^{-\beta} & e^{\beta} - \lambda \end{pmatrix} = \left(e^{\beta} - \lambda\right)^2 - e^{-2\beta} \tag{1.14}$$

Which yields the eigenvalues:

$$\lambda_+ = e^{\beta} + e^{-\beta} = 2\cosh\beta$$
$$\lambda_- = e^{\beta} - e^{-\beta} = 2\sinh\beta$$

From which it follows, for any finite N:

$$Z = \lambda_+^N + \lambda_-^N = (2\cosh(\beta))^N + (2\sinh(\beta))^N \tag{1.15}$$

Computing the limit $N \to \infty$ we can see that the partition function is dominated by the largest eigenvalue (λ_+):

$$\lim_{N\to\infty} Z = \lim_{N\to\infty} \lambda_+^N \left[1 + \left(\frac{\lambda_-}{\lambda_+}\right)^N\right] \sim \lambda_+^N \tag{1.16}$$

since $\tanh \beta^N \to 0$ as $N \to \infty$ for every $T > 0$.

And thus, going back to the free energy (equation 1.5) we can see that it is analytic for every $T > 0$, and thus the one dimensional Ising model does not exhibit a phase transition at any finite temperature.

Some assumptions can be made to understand how the magnetization function should behave in the 1D case: Since F is analytic for every $T > 0$, the magnetization function has to be so as well. Since $F(T, h) = F(T, -h)$, using the definition 1.6 it follows that $M(T, h) = -M(T, -h)$, and thus $M(T, 0) = 0$ for every $T > 0$. The last constraint is that the magnetization has to collapse to $M = \pm 1$ at exactly $T = 0$, since the system has no thermal fluctuations at absolute zero.

These considerations illustrate the picture of a function which is zero everywhere except at $T = 0$, where it jumps to $|M| = 1$, and somehow smoothly connects the two values. This is consistent with the numerical results, as will be shown in chapter 3.

1.3.2 Two dimensional case

The 2D case is far more complicated. It is also based on the transfer matrix method, but the transfer matrix is now of size $2^L \times 2^L$, where L is the side length of the lattice. Other methods have been developed to solve this problem that offer less computational complexity, but they are much more complicated. Both methods are beyond the scope of this text, and will not be discussed here.

Onsager's solution is that the critical temperature is given by the solution of the equation:

$$\sinh\left(\frac{2}{T_C}\right) = 1 \tag{1.17}$$

Which yields $T_C \approx 2.269$.

Since the 2D case is not analytically solved with $h \neq 0$, the magnetization function cannot be inferred from the free energy the same way it can be in the 1D case. Again, the methods to solve the 2D case are beyond the scope of this text, and will not be discussed here. The magnetization function is known to be given by the following expression:

$$M(T) = \begin{cases} |1 - \sinh^{-4}\left(\frac{2}{T}\right)|^{\frac{1}{8}} & T < T_C \\ 0 & T \geq T_C \end{cases} \tag{1.18}$$

which is very close to a step function. This of course makes sense, since the order state of the system has to either be $|M| = 1$ for $T < T_C$ or $M = 0$ for $T > T_C$. Note that this equation is the source of the condition 1.18 for T_C.

This also is consistent with the numerical results, as will be shown in chapter 3.

1.3.3 Three dimensional case

As stated, the solution to the 3D case of the Ising model is still an open problem in statistical mechanics. The critical temperature has been estimated to be $T_C \approx 4.5115$ by numerical methods, but no analytical solution has been found yet.

Clearly, no analytical expression for the magnetization function can be given.

Some information can be assumed from physical principles ($|M| = 1$ for $T \ll T_C$ or that $M = 0$ for $T \gg T_C$), but the exact shape of the function is unknown. What can be useful to compare numerical data with is a result from the renormalization group theory, which states that the magnetization function has to follow a power law in the vicinity of the critical temperature:

$$M(T) \sim |T - T_C|^{\beta} \quad \text{for } T \to T_C^- \tag{1.19}$$

where β is a critical exponent that has been estimated to be $\beta \approx 0.326$ by numerical methods.

Again, the numerical results are consistent with this behavior, as will be shown in chapter 3.

Chapter 2

Methods

2.1 Data representation

The code used in this project was written in python, using the Numba just-in-time compiler to achieve optimal execution speed (along best practices and algorithm optimizations in writing the code). From a development standpoint, every single Ising model configuration is represented in python as a d dimensional numpy array of shape N repeated d times.

```
1
2 def new_random_ising(N: tuple[int, ...]):
3     return np.ones(N, dtype = np.int8)
4
5 m = new_random_ising(tuple([N] * dim))
6
```

Note that the data type that the numpy array contains is np.int8. In order to minimize storage space, the idea of using boolean variables to represent 1 and -1 was considered, but was ultimately ditched in favour of storing integers. The latter grants faster executions when calculating physical observables (e.g.: energy, magnetization) at the loss of some disk space.

Every array representing a single Ising model configuration is generated and stored in another numpy array of arbitrary length that represents the Markov Chain. This is done for a number of temperatures around the expected critical temperature, and is repeated for every lattice size under scrutiny and for every dimension. Ultimately, this data is stored in a hdf5 file for efficient i/o operations and in order to contextually add metadata and processed data (e.g.: autocorrelation time, thermalisation time, etc.) to the raw chain.

For every combination of dimension and lattice size, the generated raw data is stored

in a numpy array of shape (N_{temps}, N_{steps}, $[N_{dim}] \times d$). It's worth mentioning that for every lattice in every dimension periodic boundary conditions are used, which means that the first and last spins of the lattice are considered to be neighbours. This is obviously done to avoid edge effects as much as to simplify the handling of the data, as it allows using the same code for every spin in the lattice without having to consider special cases for the spins on the edges.

A problem that arose during the development of the code was that the amount of data generated was too big to allocate in memory at once during runtime (the python scripts were run on a personal computer with 16GB of RAM). The solution to this problem was to chunk the data into smaller parts at runtime, and modify the code to load in memory these chunks, process them to obtain a partial result, discard the processed chunk, and then load the next to update the partial result until the entire dataset was processed.

This allowed to process datasets of arbitrary size without running out of memory with negligible computation time overhead.

Partially because of this choice, but more in general to allow for efficient i/o operations, the compression algorithm chosen for the hdf5 files was lzf, which is a lossless compression algorithm that is very fast and has a good compression ratio.

2.2 Monte Carlo Markov Chains

2.2.1 Generation

To study a system at a given temperature it is necessary to be able to generate a random configuration of said system. These random configurations are distributed as a function of their energy: as discussed the probability of a system of being in a state with energy E in the canonical ensemble is given by:

$$P(E) = \frac{\Omega(E)e^{-\beta E}}{Z} \tag{2.1}$$

Now, it would be ideal to write this probability analytically to be able to compute it directly and use simple sampling techniques: to do that we would need to know how to write both $\Omega(E)$ and Z, but such an operation is, unfortunately, simply not possible. The sole computing of Z would require summing over all possible configurations of the system, which in turn would require computing the energy of every single configuration. The number of possible configurations of a lattice with N^d spins is 2^{N^d} and computing the energy of a single configuration is an $O(N^d)$ operation, which means that the overall complexity of computing Z is $\mathcal{O}(N^d 2^{N^d})$, which is clearly not viable for any reasonable value of N.

The solution to this problem is a Markov Chain Monte Carlo method, that is a method for generating a sequence of independent samples from a desired distribution without knowing the normalization constant. One algorithm that produces such a chain is the Metropolis-Hastings algorithm. This algorithm can draw samples from any probability distribution with probability density $P(x)$, provided that we know another function proportional to the density P that we're able to compute analytically (in this case $e^{-\beta E}$) so that we can write $P(E) \propto e^{-\beta E}$.

The algorithm's idea is to start from a random configuration of the system and then to move to another configuration of the system with a set probability which is determined by the energy distribution.

The formal steps of the Metropolis algorithm in this use case are:

1. Start from a random configuration of the system x.
2. Randomly select a spin (or a number of spins) and flip it. We call this new configuration x'.
3. Compute the energy difference of the two systems ΔE.
4. Set an acceptance probability parameter α based on the energy change and the temperature: $\alpha = \frac{T(x' \to x)P(x')}{T(x \to x')P(x)}$ where P is the Boltzmann distribution. This is the core of the algorithm and it's what allows the Markov Chain to converge to the desired distribution.
5. Accept the new configuration with probability $p = \min(1, \alpha)$, otherwise reject it and keep the current configuration. That is, if the new configuration is more probable than the current one, accept it, otherwise accept it with probability α (that is, a probability proportional to the ratio of the two probabilities).
6. Add the current configuration to the Markov Chain.
7. Repeat steps 2-6 for a number of iterations. The elements of the chain after enough iterations are distributed as $e^{-\beta E}$

The "trick" that makes this algorithm work is that computing α the ratio of the Boltzmann factors is sufficient, as the normalization constants Z cancel out.

Note that randomly selecting a spin and flipping it (step 2) is effectively moving to another random configuration of the system close to the current one. This movement is interpreted as a random sample from a distribution centered around the current configuration. Picking a random spin to flip is an arbitrary choice, and other methods could be used to select a neighboring configuration, but has the advantage that the probability of moving from x to x' ($T(x \to x')$) is the same as moving from x' to x ($T(x' \to x)$), which is the Hastings condition of the algorithm. This condition is not striclty necessary, but it simplifies the computation of $\alpha = \frac{e^{-\beta E'}}{e^{-\beta E}} = e^{-\beta \Delta E}$ to be only dependent on the energy delta and temperature.

```python

def _metropolis_ising_1d(model: np.ndarray, T: float, steps: int):
    '''
    1D version of Metropolis-Hastings for Ising model.
    '''

    curr_model = np.copy(model)
    models_samples = np.empty((steps,) + curr_model.shape, dtype = curr_model.dtype)
    N = curr_model.shape[0]

    for i in range(steps):
        idx0 = np.random.randint(0, N)
        spin = curr_model[idx0]
        forward = (idx0 + 1) % N
        backward = (idx0 - 1) % N
        deltaE = 2.0 * spin * (curr_model[forward] + curr_model[backward])

        accept_probability = np.minimum(1.0, maxwell_boltzmann_statistics(deltaE, beta=1.0 / T))
        if np.random.random() <= accept_probability:
            curr_model[idx0] = -curr_model[idx0]

        models_samples[i] = curr_model

    return models_samples

```

Also note that since α depends only on the relative energy change the computation can be greatly optimized by computing only the energy change given by the single spin flip, instead of computing the energy of two entire configurations.

As mentioned, this algorithm can work by selecting a number of spins to flip at each proposal. In this project, only the single spin flip method was explored, but the code is written in a way that allows for easy extension to multiple spin flips.

Another arbitrary choice is the starting configuration of the system: two choices make sense and were explored in this project, that is, starting from a random configuration (hot start) or starting from a configuration with all spins aligned (cold start). The former has the advantage of being unbiased, while the latter has the advantage of being closer to the expected configuration at low temperatures, which means that the chain will thermalize faster. Thermalization is discussed properly in the next section, but in short is the process by which the Markov Chain "forgets" its initial configuration and converges to a stationary distribution. Since in practice the Markov Chains were constructed starting from low temperatures and then using the last configuration of the chain as the starting configuration for the next temperature, the cold start was used in this project

as it granted a practically thermalized state from the very beginning. Of course, this does not mean that no configuration is ever discarded, but this grants the certainty that the chain does not have thermalization issues for the first few temperatures.

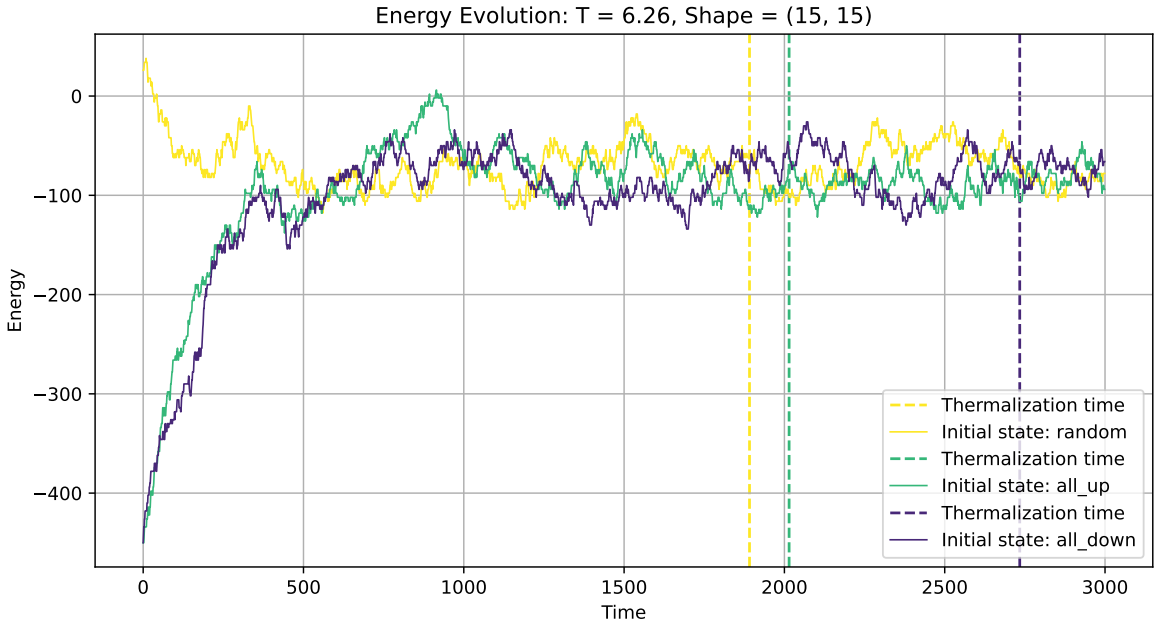


Figure 2.1: Example of some Markov Chain of configurations in the energy space.

In figure 2.1 some of the first steps of the chain can be seen as starting from different configurations (every spin set at random, all spins up, all spins down). Note that all of the chains start far from what should be the center of the distribution. This assures that before a single sample can be considered randomly sampled from the target distribution, some "adjusting" time needs to pass. This amount of time is called thermalisation time (or burn-in or equilibration time) and one estimation for this quantity is computed at runtime with techniques discussed later and shown as the dashed lines in the plot.

More generally, such a chain suffers from some kind of "recency bias", meaning that, even though the exact previous step at any point of the chain is not ever identifiable, the approximate location (in energy) in of the configuration needs some time to be allowed to properly change. This can be seen in the graphs in those locations where the curve lingers in the same vertical area for a long amount of time, and it's, of course, a random fluctuation.

Figure 2.2 shows the very same chains after a much larger number of steps. It's clear that the chains have converged to a stationary distribution, and that the initial configuration is no longer relevant.

The energy distribution of the elements in the chain can now be appreciated to be converging to a normal distribution. This is not in contradiction with the fact that the

distribution is supposed to be a Boltzmann distribution, but is in fact a consequence of the Central Limit Theorem (CLT), which for temperatures far from the critical temperature is applicable. On the contrary, close to T_C the CLT is not applicable and the variance of the distribution diverges.

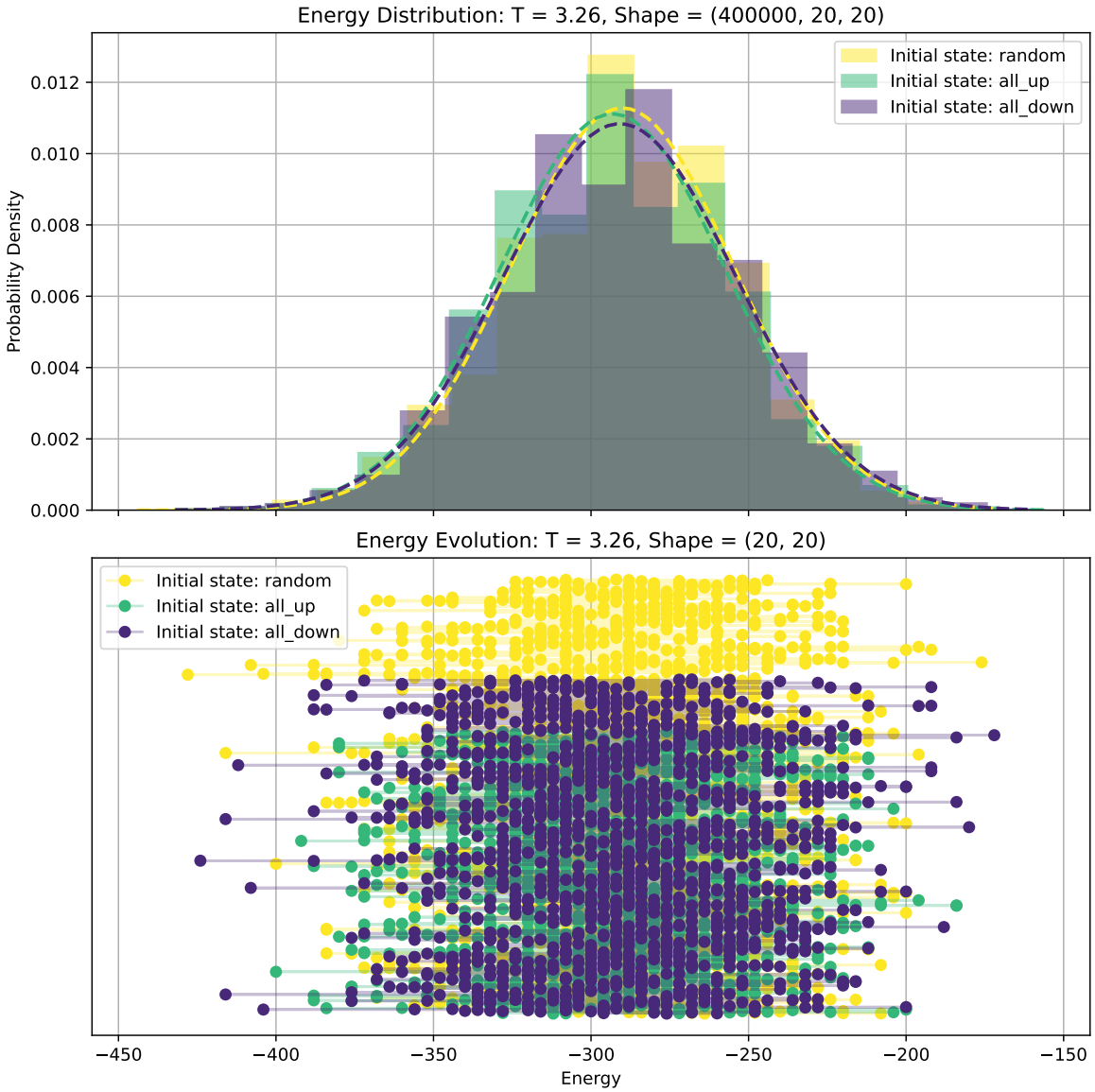


Figure 2.2: Markov Chain and relative distribution of the energy of the system.

Three things are to be noted about figure 2.2:

- Not every element of the chain is shown, but only some of them selected at regular intervals. This is just for visualization purposes, and the argument holds for the entire chain.
- Not every energy level is accessible to the system, as discretized energy levels can be seen in the vertical direction. This is obviously a consequence of the fact that the energy of the system is quantized.

- The chain starting from a random configuration is significantly longer than the other two chains. This is because plotting the chains the thermalisation times were taken into the account (with techniques discussed in the next sections), and removed. The chain starting from a random configuration has a much shorter thermalisation time than the other two chains, which is expected since the initial configuration of the other two is much further away from the stationary distribution.

2.2.2 Analysis

As stated, a Markov Chain suffers from a recency bias, and the set of configurations of the chain is not very useful as a real sample set since the elements are not independent: on the contrary, they are very closely correlated and the presence of correlations between samples would lead to biased results during analysis.

The extraction of decorrelated data is a problem composed of two parts: the thermalisation and the autocorrelation time. The first one is the number of steps that the Markov Chain needs to reach a stationary distribution, while the second one is the number of steps that need to exist between successive samples before they can be considered independent.

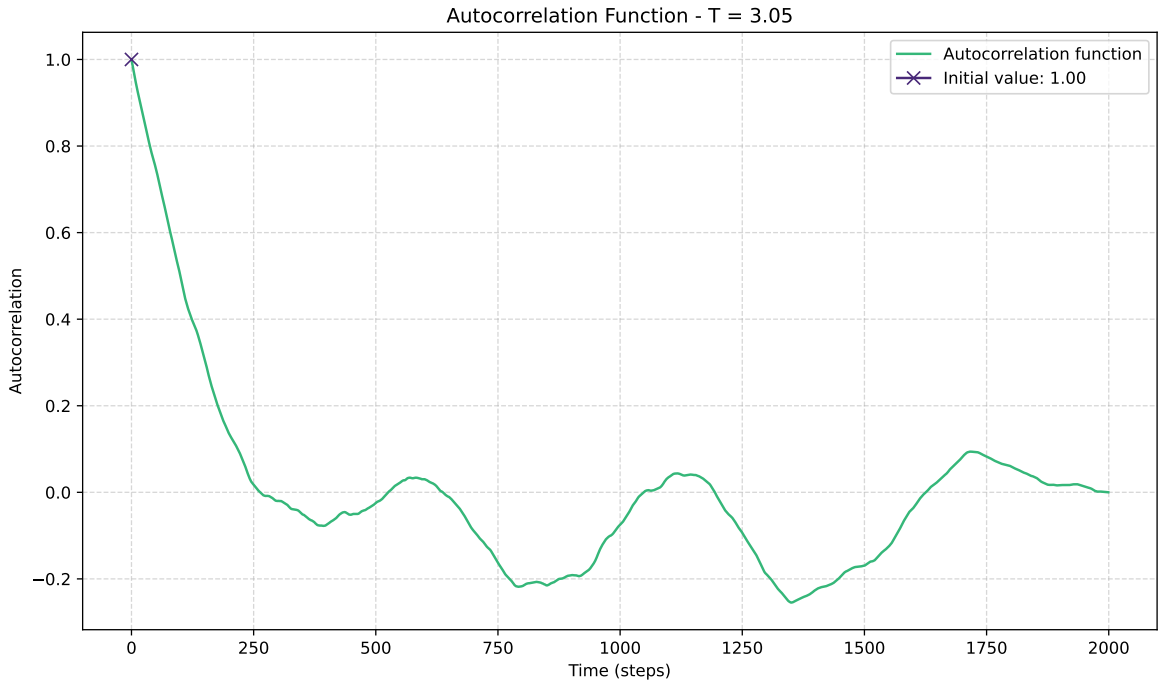


Figure 2.3: Example of autocorrelation function (of the energy) for a Markov Chain.

The plot in figure 2.3 shows the autocorrelation function of one Markov Chain. The autocorrelation function can be defined as:

$$\rho(\tau) = \frac{\mathbb{E}[(X_{t+\tau} - \mu)\overline{(X_t - \mu)}]}{\sigma^2} \tag{2.2}$$

where $\mathbb{E}$ denotes the expected value and X are the observable values measured on the configurations.

The following is the code used to estimate ρ from the Markov Chain:

```
1
2  def autocorrelation(t, observables: np.ndarray):
3          '''
4          Computes the autocorrelation function's value given:
5          - A time (step) t
6          - An array of observable values along the Markov chain
7          '''
8
9          N = len(observables)
10         if t >= N or t < 0:
11                 return 0.0
12
13         var = np.var(observables)
14
15         if var == 0:
16                 return 0.0
17
18         centered = observables - np.mean(observables)
19         autocov = np.sum(centered[:N - t] * centered[t:]) / N
20
21         return autocov / var
22
```

and it's computed with the alternative formula shown in the code snippet for performance's sake.

The function ρ quantifies the similarity between observations of a random variable (energy, in this case) at different points in time, which can be used to develop methods to extract uncorrelated samples. Specifically, we see that ρ needs a transient time to reach a state of close-to-zero values and after this time it remains close to zero.

Two parameters can be defined from the autocorrelation function:

- The exponential autocorrelation time τ_{exp}, which is the time it takes for the autocorrelation function to decay into the equilibrium region.
- The integrated autocorrelation time τ_{int}, which is the sum of the autocorrelation function over all time lags, which is a measure of the overall correlation in the data. It's defined as

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{t=1}^{\infty} \rho(t)$$

.

To sample uncorrelated data from the Markov Chain two numbers are needed:

- The number of steps to discard in front of the chain. This represents the thermalisation time, that is, the number of steps that the chain needs to reach a configuration that is representative of the stationary distribution at a certain temperature, in other words, the number of steps that the chain needs to "forget" its initial configuration. Conceptually, this is related to τ_{exp}.
- The number of steps to skip between samples (to avoid correlation between samples). This number is related to τ_{int}, and is usually set to a multiple of it (e.g.: $2\tau_{\text{int}}$).

A number of methods exist to estimate these two numbers, in this project only two are explored.

Note that theoretically speaking τ_{exp} can be neglected if the chain is long enough, that is, if we sample from a high enough multiple of τ_{int} (e.g. $1000\tau_{\text{int}}$), because the exponential decay will have already occurred. However, in practice it is always better to discard a number of steps in front of the chain to avoid the initial transient behavior. Discarding too many steps can be a problem too though, because it can lead to a loss of information and a decrease in the effective sample size (and relative errors). Practically, the number of steps to discard in this experiments will be set to $20\tau_{\text{int}}$, which is a somewhat stable estimate of τ_{exp}. Dedicated techniques to study τ_{exp} itself will not be explored.

Sokal's method

A. Sokal describes a method to estimate the integrated autocorrelation time τ_{int} in his lecture notes "Monte Carlo Methods in Statistical Mechanics: Foundations and New Algorithms" (1996). The following section discusses the method and its implementation in this project.

The natural estimator of τ_{int} would seem to be

$$\widehat{\tau}_{\text{int}} \equiv \frac{1}{2} + \sum_{t=1}^{n} \widehat{\rho}(t) \tag{2.3}$$

but this is not really the case, because this estimator has a variance that does not go to zero as the sample size n goes to infinity, so it cannot be a good estimator of τ_{int}. Qualitatively speaking, this happens because the sample autocorrelations $\widehat{\rho}(t)$ for big t contain much noise but little signal, and there are so many of them (order n) that the noise adds up to a total variance of order 1. The solution is to cut off the sum using a window $\lambda(t)$ which is ≈ 1 for small t but ≈ 0 for large t:

$$\widehat{\tau}_{\text{int}} \equiv \frac{1}{2} + \sum_{t=1}^{n} \lambda(t)\widehat{\rho}(t). \tag{2.4}$$

This retains most of the signal but discards most of the noise. A good choice is the rectangular window

$$\lambda(t) = \begin{cases} 1 & \text{if } t \leq M \\ 0 & \text{if } t > M \end{cases} \tag{2.5}$$

where M is an arbitrarily chosen cutoff. The choice of M is a tradeoff between bias and variance: if M is too small, we throw away too much signal and the estimator is biased; if M is too large, we throw away too little noise and the estimator has a large variance. A good choice would be, for example, to take M to be the first t for which $\widehat{\rho}(t)$ is negative.

The method implemented in the code is a self-consistent windowing method, where the window size is determined dynamically at runtime. The idea is to choose a window size M that is proportional to the estimated integrated autocorrelation time τ_{int}, specifically $M = c\tau_{\text{int}}$ for some constant c. This allows the window size to adapt to the data, providing a balance between bias and variance in the estimator.

```
1
2  def tau_int_sokal(observables, c = 15.0):
3      '''
4      Computes the integrated autocorrelation time using a
       ↪  self-consistent windowing method.
5      c: The windowing parameter. The bigger c, the longer the window.
6          longer window: tau has less bias
7          shorter window: tau has less variance
8      NOTE: higher c values yield more accurate results but require more
       ↪  computation time.
9      '''
10
11     N = len(observables)
12     var = np.var(observables)
13
14     if var == 0.0:
15         return 0.0
16
17     centered = observables - np.mean(observables)
18     tau = 0.5
19
20     for t in range(1, N):
21         autocov_t = np.sum(centered[:N-t] * centered[t:]) / N
22         tau += autocov_t / var
23
24         if t >= c * tau:
25             return tau
26
```

```
27      return tau
28
```

The method is called self-consistent because the window size M is determined based on the estimated value of τ_{int} itself. The algorithm iteratively computes the autocorrelation function and updates the estimate of τ_{int} until the window size M is reached. This ensures that the estimator is both efficient and accurate, as it adapts to the characteristics of the data being analyzed.

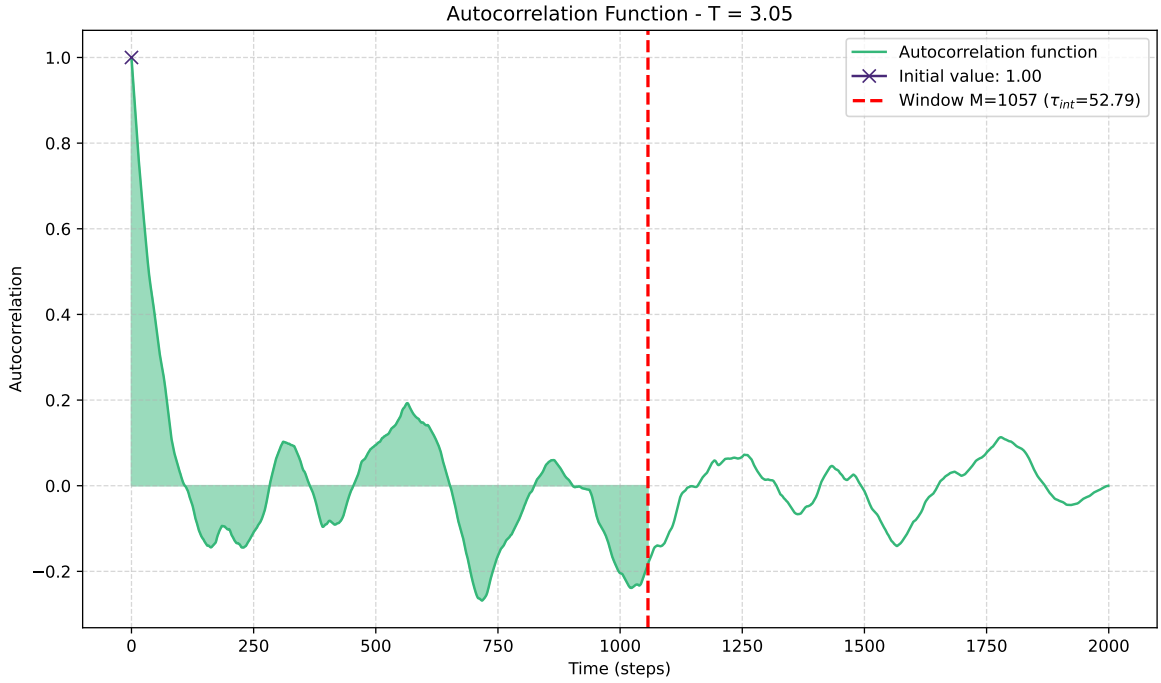


Figure 2.4: Example visualization of the windowing method.

Data blocking method

While Sokal's method is based on summing over the autocorrelation function in a particular way, the data blocking method presented in the work "Error estimates on averages of correlated data" by H. Flyvbjerg and H. G. Petersen operates in a fundamentally different way.

Let's consider a Markov Chain of length N of configurations x_i with variance σ^2 and an observable O measured on each configuration. Now suppose we simplify this Chain by averaging the values of the observable O of every consecutive k elements. This will produce a new chain of length $N_b = N/k$ of block averages $\bar{O}_i$ that are less correlated than the original chain. One can then take advantage of the law:

$$V_k = \frac{\sigma^2}{k}(1 + 2\tau_{\text{int}}) \tag{2.6}$$

to build an estimator of τ_{int}:

$$\tau_{\text{int}}(k) = k\frac{V_k}{2\sigma^2} \tag{2.7}$$

where the $-\frac{1}{2}$ term was dropped in accordance with Sokal's alternative definition of τ_{int} (even though its a negligible correction).

Note that 2.6 is intended to be a correction to the variance of the mean V_k for a chain of truly uncorrelated data:

$$V_k = \frac{1}{N_b - 1}\sum_{i=1}^{N_b}(\bar{O}_i - \bar{O})^2 = \frac{\sigma^2}{k} \tag{2.8}$$

and the correction factor can be written as a function of τ_{int} only in the limit $N \gg \tau_{\text{int}}$. To verify that these laws are consistent with each other, one can see that for truly uncorrelated data $\tau_{\text{int}} \to 0$, and so equation 2.6 reduces to equation 2.8 for $N = k$ (or $N_b = 1$ or $N \gg \tau_{\text{int}}$).

Now, the idea is that since $\tau_{\text{int}} = \tau_{\text{int}}(k)$ it can be studied as a function of k. This can be done by iterating the simplification process described above on the chain in order to create a coarser chain at every iteration, where $\bar{O_i}(k)$ are less correlated at every iteration

The blocks will contain k elements, and their size will double at every iteration. This means that for every iteration the number of blocks N_b will be halved and k will be doubled. The first iteration is $k = 1$ and $N_b = N$.

For every k the procedure is as follows:

1. Divide the chain into N_b blocks of size k. The number of blocks is then $N_b = N/k$ and compute the average of the observable O in each block. This is equivalent to averaging the observables of every two consecutive blocks of the previous iteration.
2. Compute the variance of the new set of block averages.
3. Compute $\tau_{\text{int}}(k)$ as $\tau_{\text{int}}(k) = \frac{1}{2}(k\frac{V_k}{\sigma^2} - 1)$.

The estimate $\tau_{\text{int}}(k)$ will converge to the true value of τ_{int} as k increases.

After a certain value of k the limit $k \gg \tau_{\text{int}}$ will be reached (in a numerical sense) and the estimate for τ_{int} will converge to a plateau (because V_k will be stable). The value of the plateau is then taken as the final estimate of τ_{int}.

It's important to note that this method can be used on a chain that has reached the stationary distribution, from the beginning. So, in the actual implementation, the thermalisation time is computed as $20\tau_{\text{int}}^{\text{Sokal}}$ estimated with Sokal's method, and then the data blocking method is applied to the chain after discarding the first $20\tau_{\text{int}}^{\text{Sokal}}$ steps.

The implementation is shown in the following code:

```
1
2  def tau_int_blocking(observables, burn_in = None, c_blocking = 15.0):
3      '''
4      Computes the integrated autocorrelation time using the data
       ↪  blocking method.
5      observables: ndarray of observable values along the Markov chain.
6      burn_in: int, optional
7          The number of initial samples to discard as burn-in.
8          if None, burn_in is estimated as 20 tau_int computed with the
           ↪  Sokal method.
9      '''
10
11     N = len(observables)
12
13     if burn_in is None:
14         tau0 = tau_int_sokal(observables, c = c_blocking)
15
16     observables = observables[burn_in:]
17     N = len(observables)
18
19     var = np.var(observables)
20
21     current_blocks = observables.copy()
22     tau_estimates = []
23     block_size = 1 # k
24     Nb_array = [] # Number of blocks for each block size
25     Nb = N # Number of blocks for the current block size
       ↪  (Nb_array[0])
26
27     while Nb >= 2:
28         # Estimation
29         mean_blocks = np.mean(current_blocks)
30         var_blocks = np.sum(np.power(current_blocks - mean_blocks, 2))
           ↪  / (Nb - 1)
31         tau_estimate = .5 * (block_size * (var_blocks / var) - 1)
32         tau_estimates.append(tau_estimate)
33         Nb_array.append(Nb)
34
35         # Blocking
36         if Nb % 2 != 0:
```

```
            current_blocks = current_blocks[:-1]  # Discard the last
            ↪  block if odd number of blocks
            Nb -= 1
        Nb //= 2
        current_blocks = 0.5 * (current_blocks[::2] +
        ↪  current_blocks[1::2])
        block_size *= 2

    for i in range(len(tau_estimates) - 1):
        if Nb_array[i] < 30:
            return float(tau_estimates[i])

        diff = abs(tau_estimates[i + 1] - tau_estimates[i])
        err = (tau_estimates[i] + 0.5) * np.sqrt(2.0 / (Nb_array[i] -
        ↪  1))

        if diff < err:
            return float(tau_estimates[i])

    return float(tau_estimates[-1])

```

The effect of this algorithm of the progressive smoothing of the chain can be appreciated in figure 2.5. The last plot is the one corresponding to the plateau, and the value of τ_{int} is taken from it. The other plots are just for visualization purposes, and show how the chain is progressively smoothed out by averaging every two consecutive blocks of the previous iteration.

[TODO: Add plateu detection visualization]

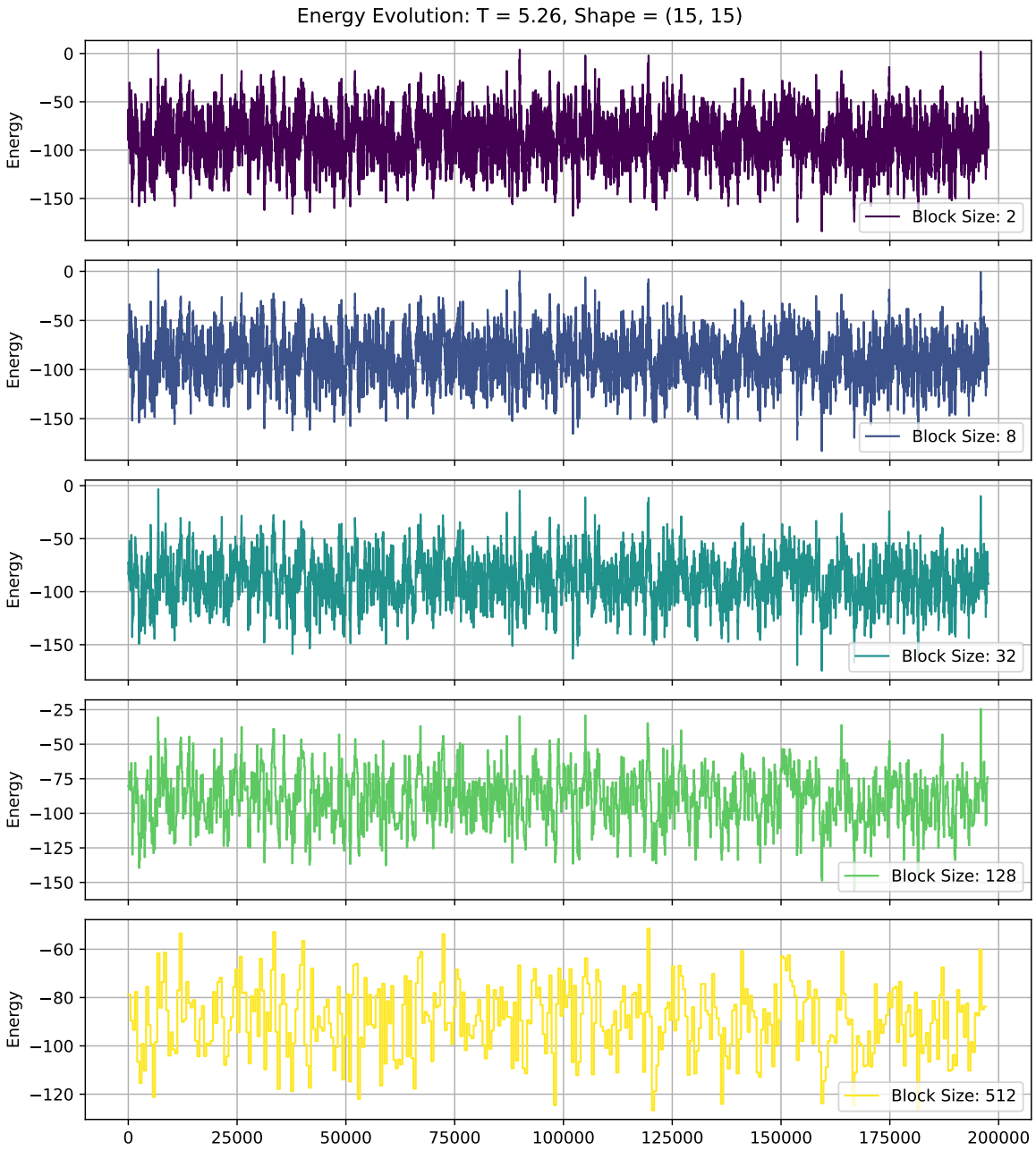
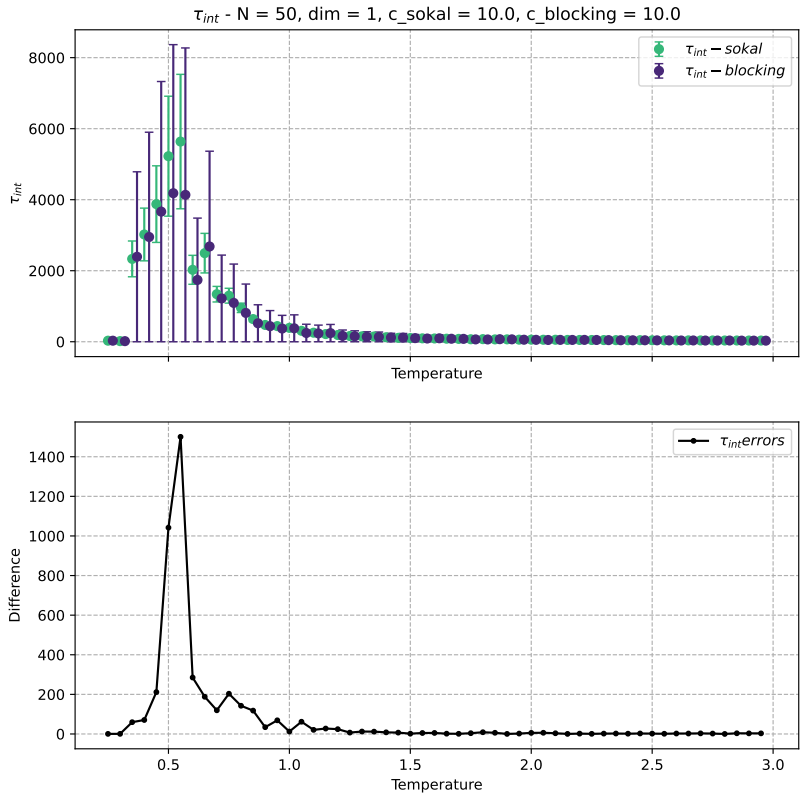


Figure 2.5: Example visualization of the data blocking method.

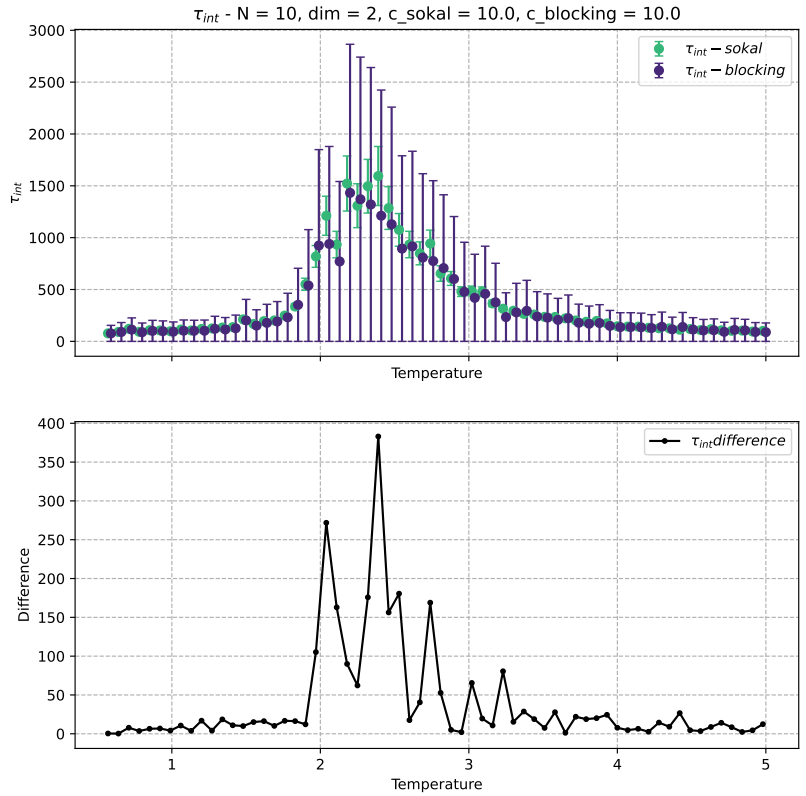
2.2.3 Comparing the two methods

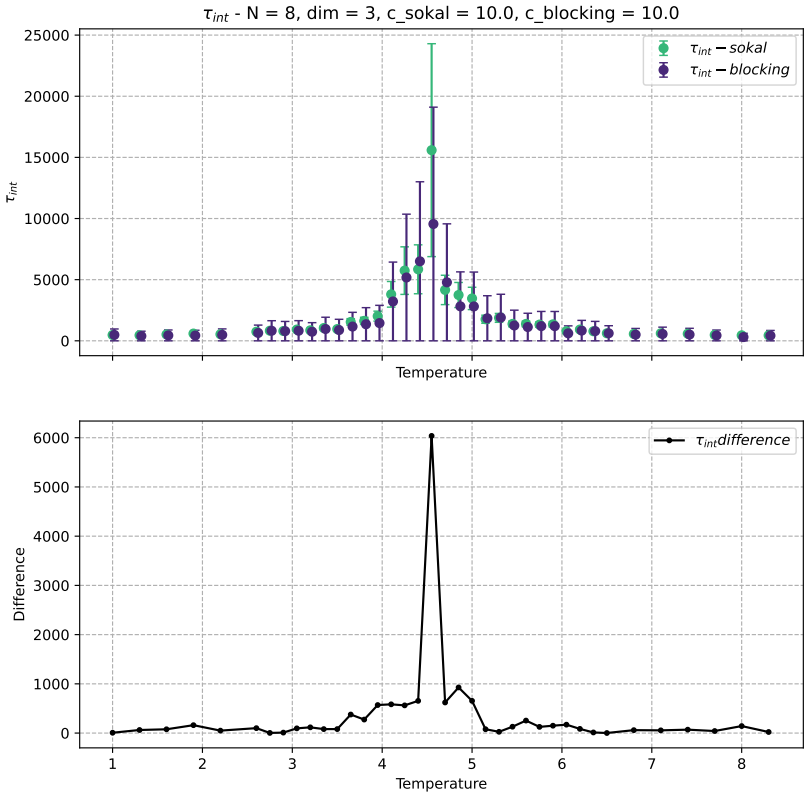
Now that both methods have been described, they can be compared. Clearly, their estimates of τ_{int} should be roughly similar. In images 2.2.3, 2.2.3 and 2.2.3 the estimates of τ_{int} for the two methods are compared for different dimensions.



The correlation between the two methods is clear, and it can be noted that both methods struggle close to T_C where this behavior is, of course, expected. This phenomenon is the very well known Critical Slowing Down, and it is a consequence of the fact that the correlation length ξ, that is, the typical distance over which spins are correlated, diverges at T_C.

Since $\tau_{\text{int}} \propto \xi^z$ and $\xi \to \infty$ at T_C, it is expected that $\tau_{\text{int}} \to \infty$ as well, and that is why these methods struggle near the critical temperature.


τ_{int} - N = 10, dim = 2, c_sokal = 10.0, c_blocking = 10.0
$\tau_{int} - sokal$
$\tau_{int} - blocking$
τ_{int}
Temperature
$\tau_{int}difference$
Difference
Temperature


τ_{int} - N = 8, dim = 3, c_sokal = 10.0, c_blocking = 10.0
$\tau_{int} - sokal$
$\tau_{int} - blocking$
τ_{int}
Temperature
$\tau_{int}difference$
Difference
Temperature

2.2.4 Improvement to the MCMC algorithm: sweeps

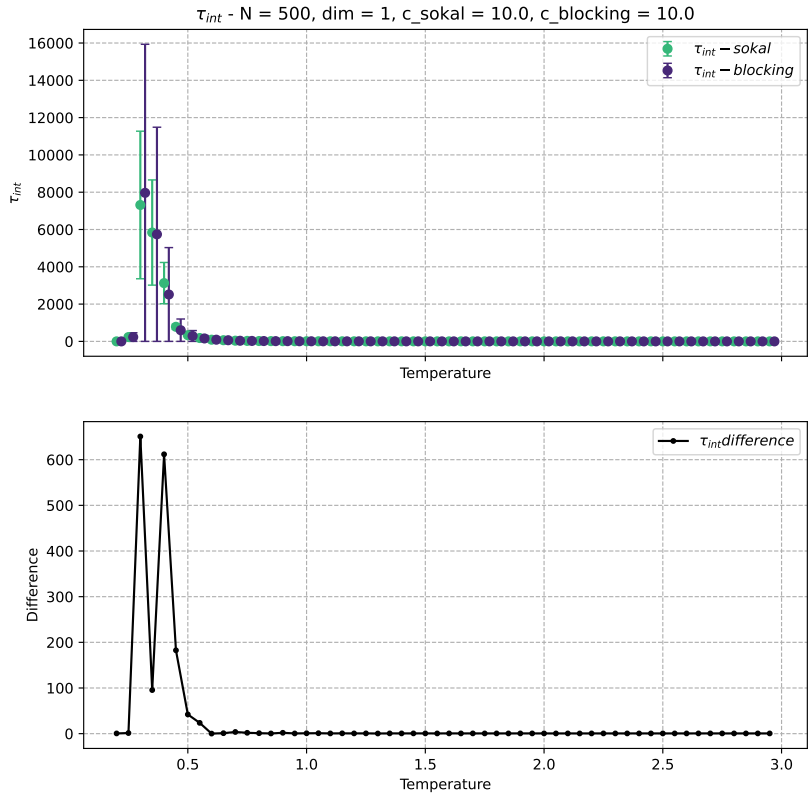
It is true that the two methods are shown to be quite compatible with each other, with produced estimations of τ_{int} that are generally in good agreement. The errors, on the other hand, are quite big, specifically close but not at T_C. This is specifically true for the data blocking method. Since the errors given by this method scale as $\sqrt{2/(N_b - 1)}$ where N_b is the number of blocks, the only way to reduce them is to increase the length of the chain.

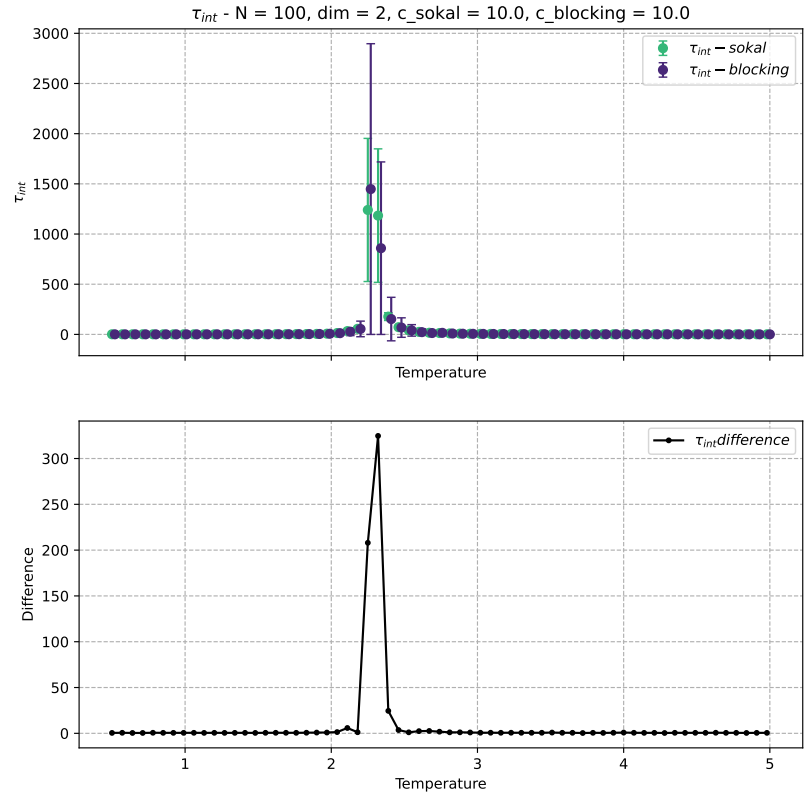
This option was not viable. To try to fix the problem, another approach was explored, that is, to change the way of building the chain. Let's define a "sweep" as the process of applying steps 2-5 of the Metropolis-Hastings algorithm (explained in Section 2.2.1) to a configuration N^d times. This is an arbitrary choice, with the rationale that a sweep is the amount of steps needed to give every spin in the lattice a chance to flip. The idea is to build the chain by adding a configuration to it only after every sweep: this ensures a lower correlation between successive configurations in the chain, and so a lower τ_{int}. This is a pretty standard approach in the literature, and is used in many implementations of the Metropolis-Hastings algorithm.

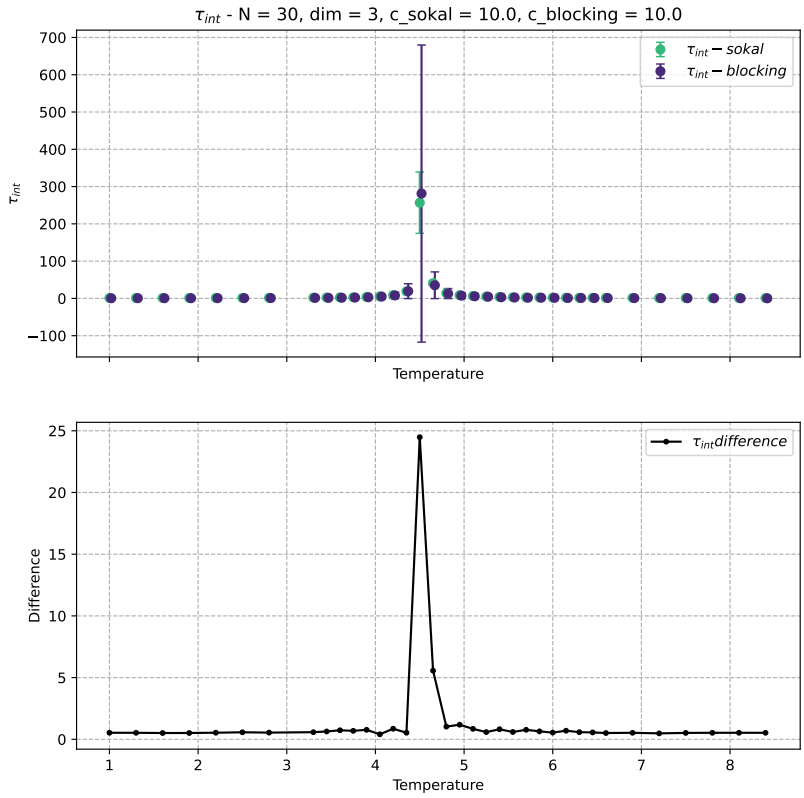
This new approach allows for much shorter chains to be used in theory, which means that more accurate results can be obtained with less computational effort. All of the production and analysis of the chains was redone with this new approach with chains with length of one less order of magnitude than before.

The results are shown in figures 2.2.4, 2.2.4 and 2.2.4. The correlation between the two methods is still clear, and the errors are significantly smaller than before. The only point where both methods fail is at the critical temperature, where, again, this is the expected behavior as the correlation length diverges and so does τ_{int}.

Every result shown in this paper from now on will be obtained with this new approach.


τ_{int} - N = 500, dim = 1, c_sokal = 10.0, c_blocking = 10.0
$\tau_{int} - sokal$
$\tau_{int} - blocking$
τ_{int}
Temperature
$\tau_{int}difference$
Difference
Temperature


τ_{int} - N = 100, dim = 2, c_sokal = 10.0, c_blocking = 10.0
$\tau_{int} - sokal$
$\tau_{int} - blocking$
τ_{int}
Temperature
$\tau_{int}difference$
Difference
Temperature


τ_{int} - N = 30, dim = 3, c_sokal = 10.0, c_blocking = 10.0
τ_{int} – sokal
τ_{int} – blocking
τ_{int}
Temperature
τ_{int}difference
Difference
Temperature

Chapter 3

Results

3.1 Introduction

After processing the data generated by the simulations to obtain the magnetizations, the critical parameter T_C can be estimated from the data.

As stated in chapter 1, while the drop in the magnetization curve suggests the temperature point at which the phase transition occurs, the numerically robust method to estimate this parameter is to look at χ. In the following sections this method will be put into practice in two different ways.

Firstly, χ will be computed directly from the magnetization data using the result of the fluctuation-dissipation theorem, which states that the susceptibility can be computed from the variance of the magnetization as a function of temperature:

$$\chi = \frac{\#\{M\}}{T}\left(\langle M^2\rangle - \langle |M|\rangle^2\right) \tag{3.1}$$

where the last term is the square of the average of the absolute value of the magnetization, which is used to avoid the cancellation of positive and negative values in the average.

Secondly, the derivative of the magnetization with respect to temperature will be computed numerically using a cubic spline interpolation, and the critical temperature will be estimated as the point at which this derivative is minimum.

The first method is theoretically sound, as it is based on the statistical properties of the magnetization, while the second method is more sensitive to noise in the data. However, both methods are expected to give similar results for sufficiently large lattices.

3.2 1D Ising model

The magnetization curves for the 1D case are presented in the following figure (3.1), which shows the magnetization $M(T)$ as a function of the temperature T for different lattice sizes N.

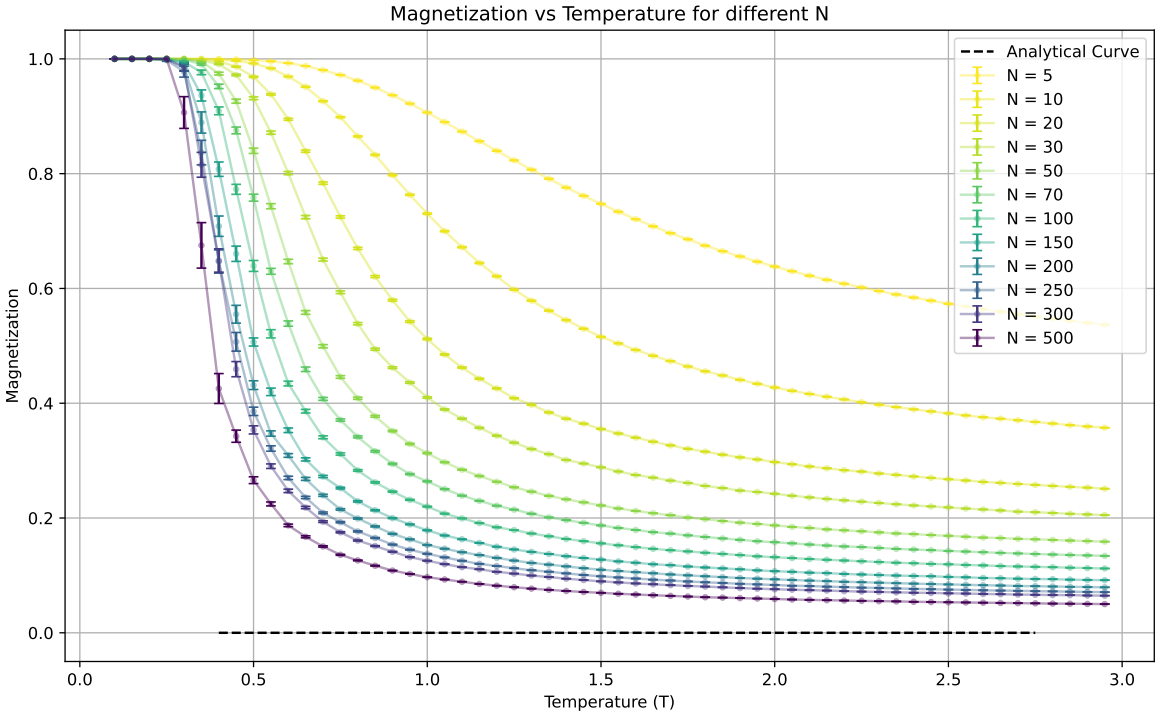


Figure 3.1: Magnetization for the 1D Ising model.

This plot is consistent with the theoretical prediction presented in chapter 1. The values close to 1 are simply an effect of finite-size corrections: the critical temperature would be exactly 0 with infinite size for every positive T.

The reason why increasing the size of the system leads to the movement of the "step" towards 0 is that the energy cost of breaking a streak of aligned spins is independent of the size of the system, which means that for bigger N the probability of having a single big streak of aligned spins is lower, and for every streak of aligned-up spins there is a high probability a streak of a same length of aligned-down spins to exist, which leads to a null magnetization. Since the flipping of a spin is an effect of thermal fluctuations the magnetization is closer to zero for higher temperatures (this can be seen in the curves of the smaller lattices: they don't drop instantly but smoothly curve towards zero), but this effect is negligible for bigger lattices, where the magnetization drops to zero faster.

Trying to extract an approximation for the critical temperature T_C from these data would be meaningless, as the expected behaviour is shown in the graphs, and an exact convergence is not achievable with a finite-size systems.

3.3 2D Ising model

The two dimensional Ising model presents a aesthetically pleasing result, as the convergence to the analytical solution is clearly visible. In figure 3.2 the magnetization $M(T)$ is shown for different lattice sizes N along Onsager's line.

The accepted value for the critical temperature of the 2D Ising model is $T_C \approx 2.269$.

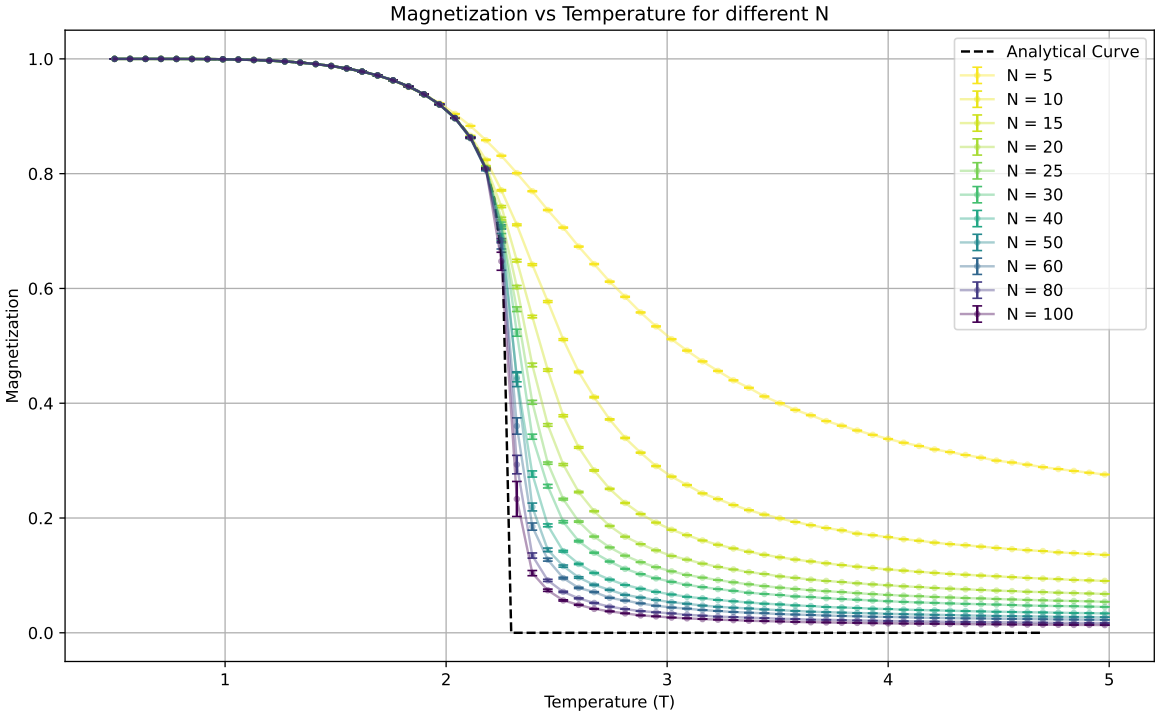


Figure 3.2: Convergence of magnetization for the 2D Ising model.

For a finite lattice the value $T_C(L)$ is an approximation for the critical temperature, and it is expected to converge to the accepted value as $L \to \infty$.

The estimation of $T_C(L)$ and is shown in figure 3.3, where the magnetic susceptibility $\chi(T)$ and the derivative of the magnetization with respect to temperature is plotted for the finest simulated lattice.

The (negative) peak of $M'(T)$ is the value of $T_C(L) \approx 2.390$, which is reasonable taking into consideration the expected relation $T_C(L) > T_C(\infty) \quad \forall\, L$. The peak of $\chi(T)$ is an the value of $T_C(L) \approx 2.460$. As stated, while $M'(T)$ gives a better result in this case, $\chi(T)$ provides a more robust estimate.

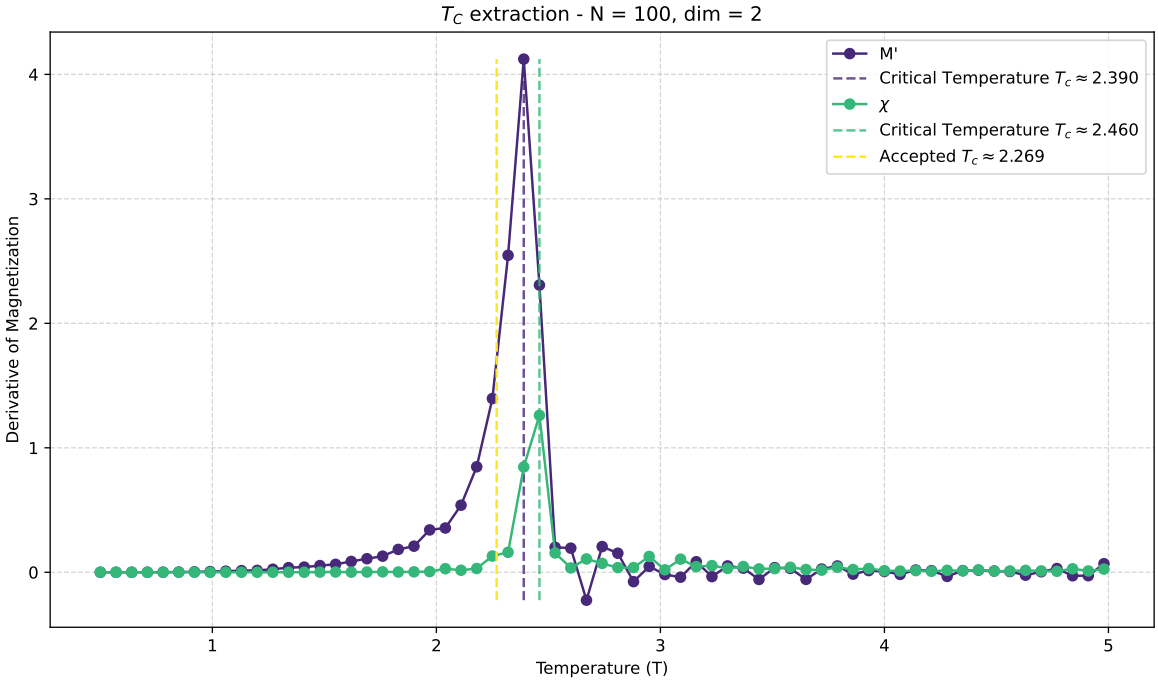


Figure 3.3: Estimation of the critical temperature (2D case).

3.4 3D Ising model

Results for the 3D Ising model are similar to the 2D case. Only, this time, in figure 3.4 the magnetization $M(T)$ is shown along the power law discussed in chapter 1.

Again, the estimation of $T_C(L)$ is shown in figure 3.5. Again, the estimations for $T_C(L)$ (≈ 4.500 for $M'(T)$ and ≈ 4.650 for $\chi(T)$) are reasonable ($T_C(\infty) \approx 4.5115$). Note that, differently from the 2D case, since the reference value is not analytically known, there is not a formal reason to expect this estimate to approach the true value from above.

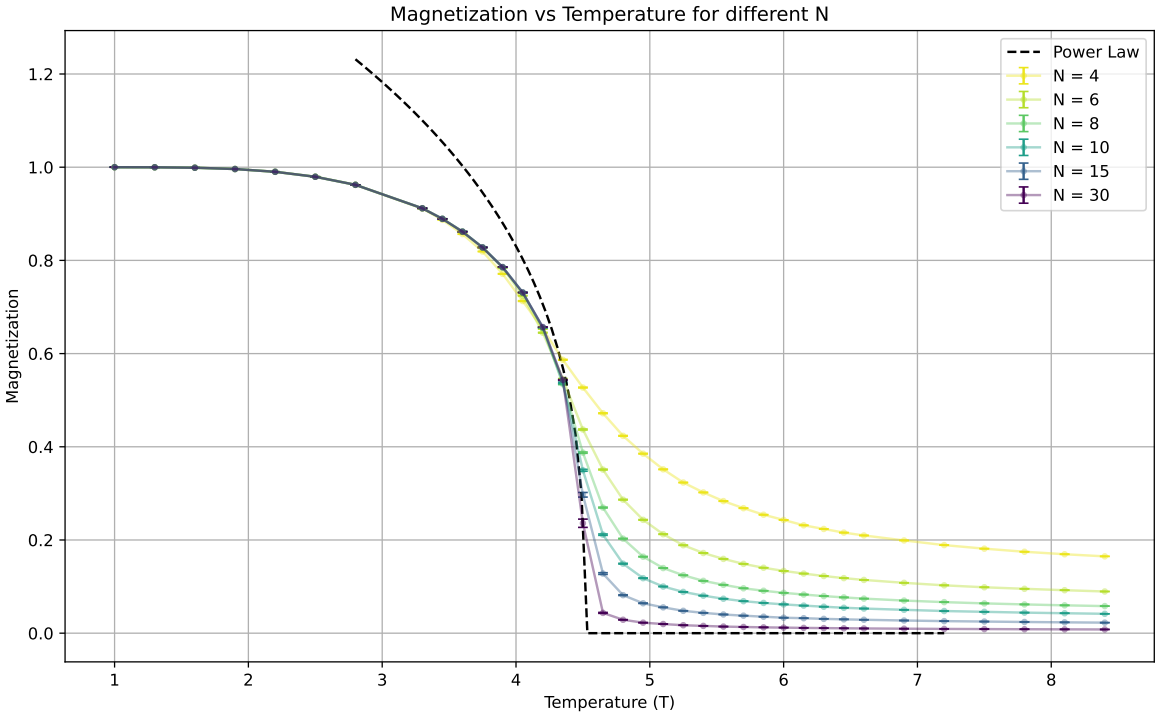


Figure 3.4: Convergence of magnetization for the 3D Ising model.

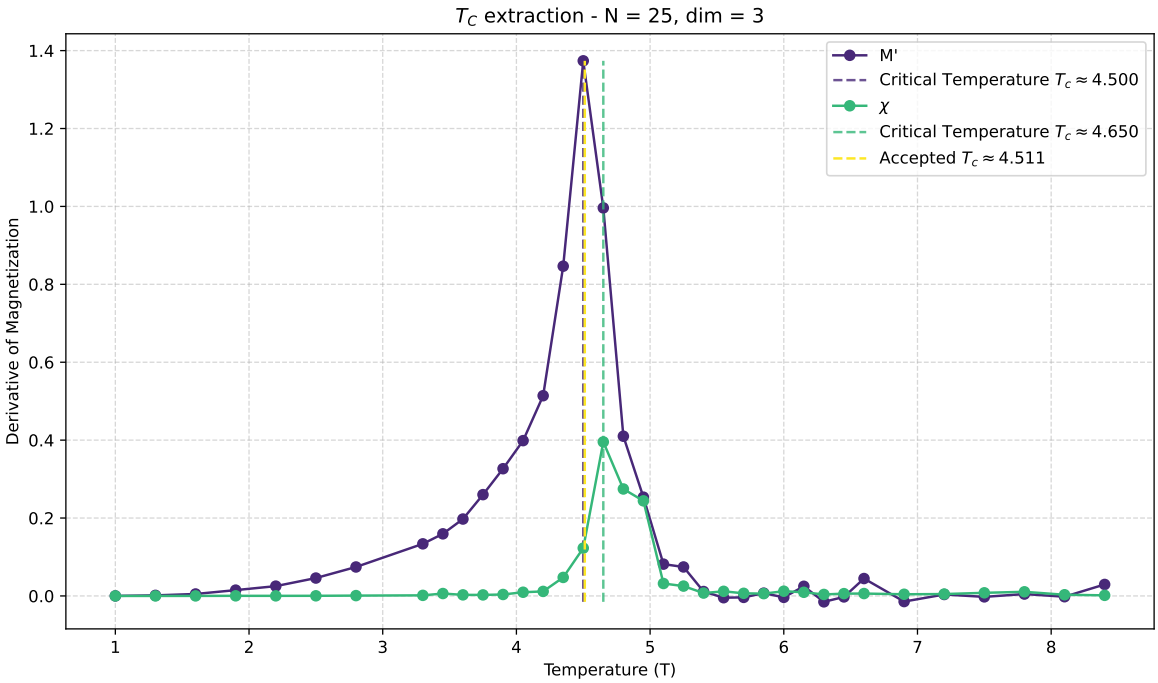


Figure 3.5: Estimation of the critical temperature (3D case).

3.5 General comments

[The difference between the 1D and 2D Ising model is that in the 1D case a single spin flip is enough to break a streak of aligned spins, that is, a single spin flip can change significantly the contribution to the magnetiziation of a section of the lattice. The equivalent of a streak of aligned spins in 2D is a cluster of aligned spins, and the equivalent of a single spin flip to break the streak is the flipping of the spins on the border of the cluster. The energy needed to flip a single spin is still independent of the size of the system, but the size of the perimeter of a cluster is not, and this introduces a size dependence in the probability of having a cluster of aligned spins, which is what leads to a threshold behavior.]

But why does a finite critical temperature exist in 2D and 3D and not in 1D?

The answer can be understood in the context of the lattice approach by thinking about the energy cost of flipping a single spin: this operation depends only on the nearest neighbours of the spin, which means that if d is fixed, the flip energy cost does not change with N.

In general, the probability of having a streak of aligned spins is high (because that is the most stable configuration of the system). Now, let's consider a 1D lattice of very coarse size (think $N = 5$). In this case, even at relatively high temperatures, a configuration of two different streaks of aligned spins is improbable, because it is not energetically favorable. This implies a non zero magnetization even at non-zero temperatures like can be seen in figure 3.6, where a white or black square is an up or down spin.

This changes with finer lattices (think $N = 100$). In this case, the probability of having a configuration with two different streaks of aligned spins is much higher, because *multiple long streaks of aligned spins are permitted*, and globally the magnetization averages out to zero. The key takeaway is that there does not exist a threshold temperature for this phenomenon, and the magnetization is always zero for sufficiently large N. This is shown in figure 3.7 with the same convention.



Figure 3.6: Coarse 1D lattice at any temperature $T > 0$.



Figure 3.7: Fine 1D lattice at any temperature $T > 0$.

[TODO: Also cite the entropic argument for the existence of a phase transition in 2D and 3D, and the absence of it in 1D. Also cite the Peierls argument more precisely.]

This logic does not hold for higher dimensions.

In 2D, the equivalent of a streak is a cluster of aligned spins, and the equivalent of a single spin flip to break the streak is the flipping of the spins on the border of the cluster. Since the border of the cluster does actually scale with the size of the system, the probability of having a cluster of aligned spins is size dependent (because the energy

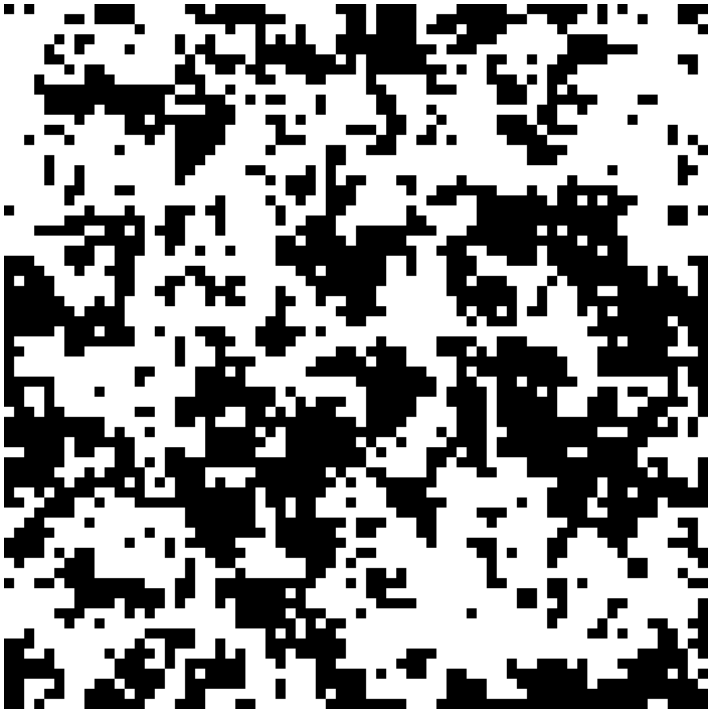


Figure 3.8: Example 2D lattice thermalized close to T_C.

cost of flipping the boundary of the cluster is), which means that there exists a minimum thermal excitation needed to break the cluster. The temperature at which this happens is the critical temperature. The same argument can be made about the 3D case, where the border of a cluster is a surface, which scales with the size of the system.

[Reorganize this section and discuss sweeps]

Appendix A

Appendix

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Bibliography

[1] Author, Title, Journal/Book, Year.